



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE SISTEMAS DE COMUNICACIONES



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Vargas Luna José Ángel

N° de Cuenta: 318252333

GRUPO DE LABORATORIO: 07

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12-04-2026

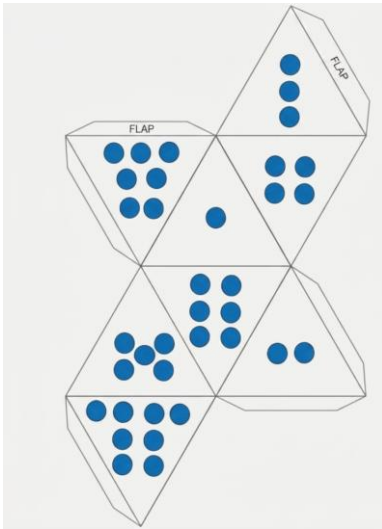
CALIFICACIÓN: _____

Ejercicio 1: Crear un dado de 8 caras y texturizarlo por medio de código.

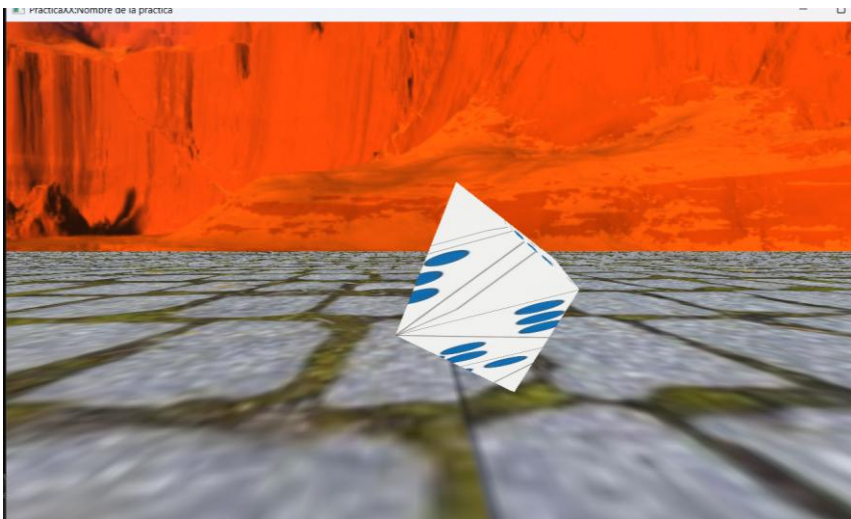
Para este apartado primero crearemos un octaedro con triángulos, para después utilizar una imagen para texturizarlo.

```
80 void CrearDado8Caras() {
81     // Formato: x, y, z, u, v, nx, ny, nz
82     // Nota: He ajustado los valores UV para que coincidan con los puntos de tu imagen
83
84     GLfloat oct_vertices[] = {
85         // --- FILA SUPERIOR (Triángulos con punta hacia ARRIBA en la textura) ---
86
87         // Cara 1 (1 punto) - Centro superior
88         0.0f, 1.0f, 0.0f, 0.458f, 0.96f, 0.577f, 0.577f, 0.577f,
89         1.0f, 0.0f, 0.0f, 0.556f, 0.53f, 0.577f, 0.577f, 0.577f,
90         0.0f, 0.0f, 1.0f, 0.359f, 0.53f, 0.577f, 0.577f, 0.577f,
91
92         // Cara 3 (3 puntos) - Derecha superior
93         0.0f, 1.0f, 0.0f, 0.742f, 0.96f, 0.577f, 0.577f, -0.577f,
94         0.0f, 0.0f, -1.0f, 0.840f, 0.53f, 0.577f, 0.577f, -0.577f,
95         1.0f, 0.0f, 0.0f, 0.643f, 0.53f, 0.577f, 0.577f, -0.577f,
96
97         // Cara 4 (4 puntos) - Extremo derecho superior
98         0.0f, 1.0f, 0.0f, 0.855f, 0.35f, -0.577f, 0.577f, -0.577f, // Reajustado por posición en net
99         -1.0f, 0.0f, 0.0f, 0.954f, 0.05f, -0.577f, 0.577f, -0.577f,
100        0.0f, 0.0f, -1.0f, 0.755f, 0.05f, -0.577f, 0.577f, -0.577f,
101
102        // Cara 7 (7 puntos) - Izquierda superior
103        0.0f, 1.0f, 0.0f, 0.260f, 0.96f, -0.577f, 0.577f, 0.577f,
104        0.0f, 0.0f, 1.0f, 0.161f, 0.53f, -0.577f, 0.577f, 0.577f,
105        -1.0f, 0.0f, 0.0f, 0.358f, 0.53f, -0.577f, 0.577f, 0.577f,
106
107        // --- FILA INFERIOR (Triángulos con punta hacia ABAJO en la textura) ---
108
109        // Cara 2 (2 puntos)
110        0.0f, -1.0f, 0.0f, 0.855f, 0.12f, -0.577f, -0.577f, -0.577f,
111        0.0f, 0.0f, -1.0f, 0.755f, 0.53f, -0.577f, -0.577f, -0.577f,
112        -1.0f, 0.0f, 0.0f, 0.954f, 0.53f, -0.577f, -0.577f, -0.577f,
113
114        // Cara 5 (5 puntos)
115        0.0f, -1.0f, 0.0f, 0.655f, 0.12f, 0.577f, -0.577f, -0.577f,
116        1.0f, 0.0f, 0.0f, 0.754f, 0.53f, 0.577f, -0.577f, -0.577f,
117        0.0f, 0.0f, -1.0f, 0.556f, 0.53f, 0.577f, -0.577f, -0.577f,
118
119        // Cara 6 (6 puntos)
120        0.0f, -1.0f, 0.0f, 0.458f, 0.12f, 0.577f, -0.577f, 0.577f,
121        0.0f, 0.0f, 1.0f, 0.359f, 0.53f, 0.577f, -0.577f, 0.577f,
122        1.0f, 0.0f, 0.0f, 0.556f, 0.53f, 0.577f, -0.577f, 0.577f,
123
124        // Cara 8 (8 puntos)
125        0.0f, -1.0f, 0.0f, 0.260f, 0.12f, -0.577f, -0.577f, 0.577f,
126        -1.0f, 0.0f, 0.0f, 0.161f, 0.53f, -0.577f, -0.577f, 0.577f,
127        0.0f, 0.0f, 1.0f, 0.359f, 0.53f, -0.577f, -0.577f, 0.577f,
128    };
129
130    unsigned int oct_indices[] = {
131        0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
132        12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23
133    };
134
135    Mesh* dado8 = new Mesh();
136    dado8->CreateMesh(oct_vertices, oct_indices, 192, 24);
137    meshList.push_back(dado8);
138 }
```

Después de crearlo, vamos a texturizarlo usando de referencia una imagen de un dado de 8 caras, el cual es este.



Dando como resultado un octaedro, sin embargo, mis triángulos en mi modelo son equiláteros mientras en mi imagen son isósceles, lo cual genera una alteración en las caras de mis dado, generando como efecto el siguiente.



Ejercicio 2: Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin)

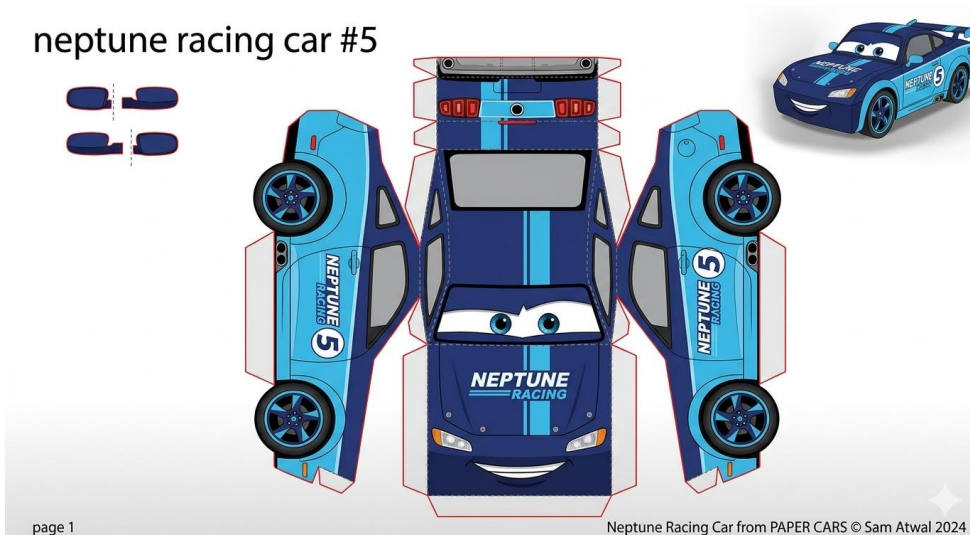
Ejercicio 3: Texturizar la cara del personaje de la imagen tipo cars en el parabrisas (ojos) y detalles en cofre y parrilla de su propio modelo de coche

Primero, le pedí a una IA que modificara mi imagen de mi coche a un modelo de versión para imprimir, lo que facilitaría mi trabajo en el texturizado.



ado.

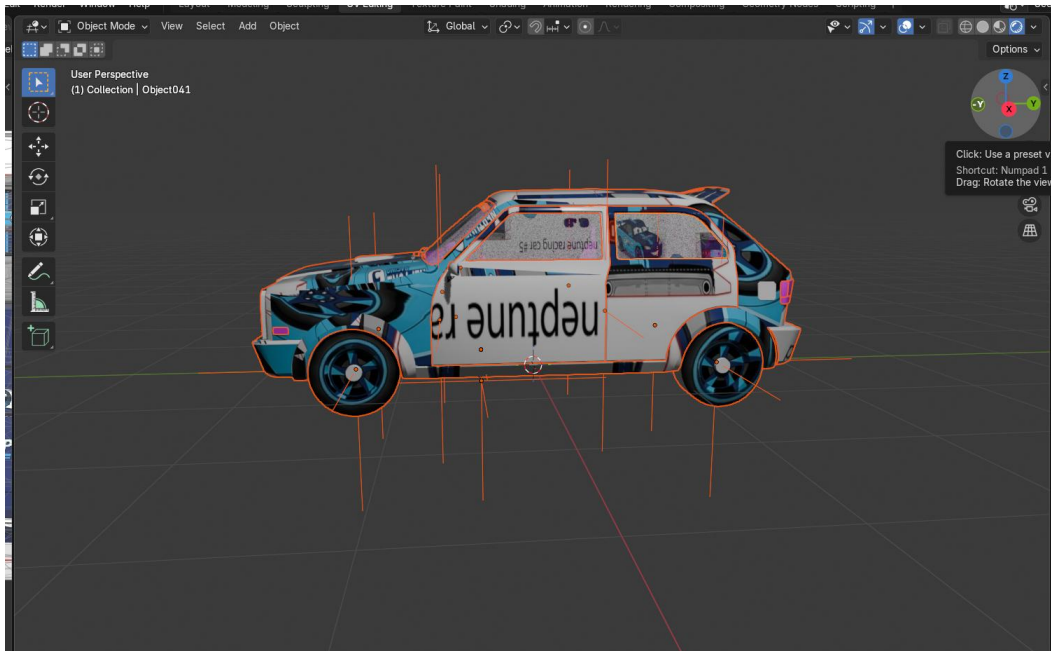
neptune racing car #5



page 1

Neptune Racing Car from PAPER CARS © Sam Atwal 2024

Este modelo lo utilicé en blender, solamente que mi modelo de coche contaba con demasiados triángulos y por ende la textura de la carrocería se ve mezclada con el resto de la imagen, lo que provoca que se vean las llantas y no se vea el parabrisas. Pero igual continúe texturizándolo hasta llegar a este apartado.



Las llantas y el rin están texturizados junto con la carrocería, un leve problema del modelo que descargué.

Para después cargar mi modelo a mi código en Visual Studio, tanto la imagen que uso como textura como el modelo de mi carro texturizado se deben encontrar dentro de mi carpeta Textures y Models, lo cual está hecho.

Finalmente, en el código principal de mi práctica adjunto tanto el modelo texturizado como la textura que se utiliza.

```

27
28     Texture pisoTexture;
29     Texture Dado_8caras;
30     Texture Textura_carro;
31
32     Model Kitt_M;
33     Model Llanta_M;
34     Skybox skybox;
35     Model Carro_texturizado_nuevo;

```

Aquí mandamos a llamar tanto a la textura del carro, como al modelo texturizado por blender.

```

158
159     Dado_8caras = Texture("Textures/Dado_8caras.png"); // Asegúrate que el nombre coincida
160     Dado_8caras.LoadTextureA();
161     Textura_carro = Texture("Textures/Textura_carro.png");
162     Textura_carro.LoadTextureA();
163     Carro_texturizado_nuevo = Model();
164     Carro_texturizado_nuevo.LoadModel("Models/Carro_texturizado_nuevo.obj");

```

En este momento cargamos tanto el modelo como la textura del mismo al programa, llamándolos desde su respectiva carpeta.

```

215
216     model = glm::mat4(1.0);
217     model = glm::translate(model, glm::vec3(-3.0, 3.0, -2.0));
218     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
219     Carro_texturizado_nuevo.RenderModel();
220
221     glUseProgram(0);
222     mainWindow.swapBuffers();
223 }
224 return 0;
225 }

```

Finalmente, en este apartado, exponemos el carro a llamarlo para que aparezca en el window y se aparezca modelado.



Sin embargo, mi modelo no permite texturas en las llantas ni el parabrisas, en el caso del parabrisas o luces se ven completamente transparente, lo cual hace que se vea negro y en el caso de las llantas solo se pueden usar colores específicos.

Problemas Obtenidos

Me costó mucho lograr obtener las coordenadas correctas del dado para su texturización. Además de que mi modelo de coche no cuenta con la suficiente optimización para que pueda ser texturizado a un cien por ciento.

Conclusiones

La práctica fue bastante útil al hacernos texturizar de dos formas distintas, además de comprender como funciona el texturizado y la implementación de las texturas con los modelos.

Referencias

Formato: Google. (2026). *Gemini* (Versión de abril) [Modelo de lenguaje extenso].

<https://gemini.google.com/>

Sellers, G., Wright, R. S., & Haemel, N. (2014). *OpenGL Superbible: Comprehensive Tutorial and Reference* (6.th ed.). Addison-Wesley Professional.

Barrett, S. (s. f.). *stb_image: Image loading/decoding from file/memory* [Software].
<https://github.com/nothings/stb>

De Vries, J. (s. f.). *Learn OpenGL: Textures*. <https://learnopengl.com/Getting-started/Textures>